

House of Pakistan — Security & Compliance Overview

Authentication & Authorization

- **JWT-based authentication** using `verifyTokenMiddleware`
 - Role-based access control (RBAC) via decoded tokens (e.g. `ADMIN`, `USER`)
 - Protected routes using `verifyToken()` middleware to enforce ownership and access
-

Data Validation & Sanitization

- **Input validation** via `Joi` schemas for all request bodies (create, update, filters)
 - Accepts only safe and required fields
 - Trims strings, handles nulls gracefully, and restricts data types (e.g. UUIDs, enums)
-

Database Security

- ORM: Prisma with PostgreSQL
 - **Prevented SQL Injection** through Prisma query APIs (e.g., no raw query concatenation)
 - Uses `@unique` and `@map` for normalized and protected schema control
-

File Upload & Storage

- **S3 integration** with strict MIME type and size checks
 - Filenames sanitized using `slugify` and timestamped
 - S3 `PutObjectCommand` ensures `ContentType` headers are set securely
-

Geospatial Data Handling

- `latLng` stored using PostGIS-compatible `geometry(Point)`
 - Accepts validated `{ lat, lng }` input only through structured APIs
-

✓ Error Handling & Logging

- Custom error class (`CustomError`) for consistent HTTP status codes and messages
 - Centralized logging with descriptive error messages for debug and audit trails
-

🔄 Data Integrity

- All `upsert` operations use `@unique` constraints to avoid duplicates
 - Related entities (e.g. categories, tags, subcategories) are validated for existence before creation
-

📁 Compliance & Best Practices

- Sensitive keys (e.g., AWS, DB) loaded from `.env` via `ENV` constants
- Only explicitly allowed fields are persisted
- All UUIDs are server-generated (`uuid()` default in Prisma)